

Unsupervised Subgraph Anomaly Detection Based on Pattern Collaboration

Jiayang Sun
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
sunjiayang@hust.edu.cn

Wei Xiang
Central China Normal University
Faculty of Artificial Intelligence in
Education
Wuhan, China
xiangwei@ccnu.edu.cn

Shenghao Liu*
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
liushenghao@hust.edu.cn

Meng Luo
Zhejiang University
The State Key Laboratory of
Blockchain and Data Security
Hangzhou, China
Hangzhou High-Tech Zone (Binjiang)
Institute of Blockchain and Data
Security
Hangzhou, China
meng.luo@zju.edu.cn

Dandan Zheng
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
yuandan.zdd@antgroup.com

Xianjun Deng
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
dengxj615@hust.edu.cn

Qiankun Zhang
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
Ministry of Education of China
Key Laboratory of Cyberspace
Security
Zhengzhou, China
qiankun@hust.edu.cn

Abstract

Subgraph Anomaly Detection (SAD) is crucial for identifying groups that deviate from the regular pattern within graphs, which benefits different domains such as financial fraud and network security. However, current studies rely on traditional node detection methods and fixed sampling strategies of subgraph structures, which makes it difficult to learn the pattern collaboration behavior of subgraphs. To address this limitation, this paper proposes a novel unsupervised framework named PC-SAD. The PC-SAD framework first employs an improved Graph AutoEncoder to identify core anomaly nodes by capturing multi-scale neighborhood information. Starting from these core anomaly nodes, we sample candidate subgraphs with path, tree, and cyclic structures, and enhance them according to the characteristics of the subgraph structures. Subsequently, candidate subgraphs are fed into the proposed Pattern Collaboration-based

Graph Contrastive Learning method to generate collaborative pattern embeddings, thereby distinguishing anomaly subgraphs. The experimental results show that PC-SAD outperforms the state-of-the-art baseline methods on four benchmark datasets, which proves that PC-SAD is an effective solution to detect anomaly subgraphs.

CCS Concepts

• **Networks** → **Topology analysis and generation**; • **Information systems** → **Social networks**.

Keywords

Subgraph Anomaly Detection; Pattern Collaboration; Graph AutoEncoder; Graph Contrastive Learning

* Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792214>

ACM Reference Format:

Jiayang Sun, Shenghao Liu, Xianjun Deng, Wei Xiang, Meng Luo, Qiankun Zhang, and Dandan Zheng. 2026. Unsupervised Subgraph Anomaly Detection Based on Pattern Collaboration. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3774904.3792214>

1 Introduction

Subgraph Anomaly Detection (SAD) plays a crucial role in financial security [7], social networks [37], and academic citation networks [12], achieving remarkable success. Its primary objective is to identify subgraphs with anomalous patterns in graph data, helping minimize the considerable damage caused by detrimental events [21]. Depending on the discriminant method of the anomaly subgraphs, SAD can be divided into two levels: Node-level (N-SAD) and Group-level (G-SAD). Specifically, N-SAD [10, 14, 22] involves clustering of anomalous nodes to form anomalous subgraphs; G-SAD [1, 34, 40] identifies anomalous nodes and samples candidate subgraphs from them, where candidate subgraphs with an anomaly score are considered anomaly subgraphs.

While G-SAD methods provide a novel perspective for subgraph anomaly detection, they fail to accurately locate core nodes (anchors) within anomaly nodes. In essence, each subgraph is generated from an anchor, but not every node can sample an anomalous subgraph. Therefore, locating anchors is crucial for both sampling and identifying subgraphs. However, current studies usually adopt a single Graph Neural Network [38] to learn the representation of nodes, and use Graph AutoEncoder [9] to distinguish anomalous representation. The former overly smooths the representation of neighboring nodes [11], which destroys the discriminant anomalous information. On the other hand, the latter tends to force an anomalous representation towards the distribution of normal nodes during the reconstruction process [36]. These two methods make it difficult to locate anchors accurately.

In addition, during the sampling process, the G-SAD methods rely on fixed sampling strategies of subgraph structures [1]. This approach is inadequate, since anomaly subgraphs often violate fixed network rules and exhibit collaborative patterns [16]. Unfortunately, existing G-SAD methods treat subgraphs independently, which neglect collaborative pattern structures. Specifically, these studies usually adopt Graph Contrastive Learning or its variants [28, 35] to learn the representation of subgraphs. However, anomalous collaborative patterns are often well hidden and overlap with normal subgraph embeddings in the latent space [30]. The limited neighborhood information makes it impossible to effectively distinguish subgraph representations. In particular, it is known that, in a blockchain network [8], multiple anomalous accounts frequently conduct small value transfers through cross-block operations in the short term. This collaborative strategy forms a hidden anomaly subgraph and breaks the fixed transaction pattern [33]. For example, Figure 1 shows a schematic diagram of such transactions.

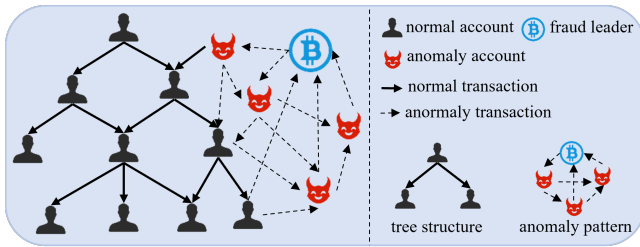


Figure 1: An anomalous collaboration pattern in the blockchain transactions.

In Figure 1, normal transactions follow a tree structure. For instance, a user makes payments to multiple merchants. In contrast, in illegal transactions, the fraud leader deposits funds into different accounts. Taking these accounts as the sources, funds are transferred frequently through some anomaly accounts, and eventually flow back to the leader. These anomaly accounts break the fixed pattern and together form a star structure. Structures like stars or others that are inconsistent with most structures in the network are called anomalous collaborative patterns. Unfortunately, existing SAD methods are incapable of learning anomalous collaboration patterns, which reduces the performance of the detection [18].

To address the above challenges, we propose a novel unsupervised PC-SAD framework. Specifically, we use the improved Multi-Scale Graph AutoEncoder to identify anchors by capturing global and local neighborhood information. This will effectively improve the detection performance of anomaly nodes. Subsequently, starting from these anchors, we sample candidate subgraphs with three structures and enhance them according to the characteristics of the subgraph structures. Pattern Collaboration-based Graph Contrastive Learning, introduced in this paper, utilizes dual-space feature extraction and a transformer to learn the representation of subgraphs. The representation contains the collaboration pattern information in the subgraph, which is then fed into an anomaly detector to compute anomaly scores. The main contributions of this paper are summarized as follows.

- We propose a novel unsupervised method PC-SAD. PC-SAD has significant applications, particularly in the financial security and paper citation network.
- We propose a Multi-Scale Anomaly Node Detection module that can accurately identify anchors.
- We also propose a Pattern Collaboration-based Graph Contrastive Learning module that can effectively extract collaborative patterns.
- Experiments conducted on four datasets demonstrate that PC-SAD achieves significant performance compared to the state-of-the-art N-SAD and G-SAD methods.

2 Related Work

2.1 Subgraph Anomaly Detection

Subgraph Anomaly Detection originates from the research domain of Graph Anomaly Detection, and excellent reviews of this literature are available in [21, 25, 26]. Existing approaches can be roughly classified into two main categories: Node-level methods (N-SAD) and Group-level methods (G-SAD).

Traditional N-SAD rely on clustering techniques to quantify subgraph anomalies [4, 27]. Effective for small-scale graphs, they degrade in large networks due to exponential subgraph space growth [42]. With the rise of advanced Deep Learning, Graph Neural Network and Graph AutoEncoder have emerged as a dominant framework for SAD. Graph Neural Network-based methods, such as Implicit Anomaly Subgraph Detection [29], which learns latent representations of subgraphs and detects anomalies by measuring reconstruction errors. Other works, like Federated Graph Anomaly Detection [19], combine node anomaly scores to infer subgraph anomalies. However, these methods often struggle with capturing

collaborative patterns in subgraphs, as they primarily focus on individual node or edge anomalies rather than collective behaviors.

Compared to widely studied N-SAD, Subgraph Anomaly Detection based on Group-level is a recently emerging research direction and receives much less attention. We currently know there Group-level methods: Deep Structure Learning [34], Anomalous Subgraph Autoencoder [40], Topology Pattern Enhanced [1]. They first utilize Graph AutoEncode to detect anomaly nodes, then sample subgraphs from the anomaly nodes, and calculate an anomaly score for each subgraph. Nevertheless, the subgraph with collaborative patterns remains underexplored. Existing Group-level methods often rely on fixed sampling strategies that fail to model the subtle inherent in real-world anomalies, such as blockchain fraud schemes or cross-blockchain transaction networks.

2.2 Graph Contrastive Learning

Graph Contrastive Learning is a popular method for discriminative representation learning in graph anomaly detection [15]. For example, Hassani et al. [32] introduced a mutual information maximization strategy to distinguish anomaly nodes by comparing local-global structures, laying the groundwork for anomaly detection. However, these methods were not explicitly designed for subgraphs, as they overlook subgraph structural hierarchy.

Recent efforts have extended Graph Contrastive Learning to SAD. For example, Hu et al. [13] propose a multi-view contrastive framework that aligns subgraph representations across different neighborhood scales, enabling the capture of both local and global structural. By contrasting augmented subgraph pairs, Fu et al. [17] and Qin et al. [24] distinguish normal subgraphs from anomalous deviations. In addition, Suresh et al. [31] and Yu et al. [39] fed the Graph Information Bottleneck (GIB) into Graph Contrastive Learning, which aims to decrease the mutual information between the candidate subgraph and its augmentation view. Despite these advancements, existing frameworks focus on pairwise contrastive objectives, which struggle to capture higher-hop representations in subgraphs.

3 Method

In this section, we introduce our proposed framework PC-SAD. We provide a problem formulation (Sec. 3.1), followed by detailed explanations of three modules: multi-scale anchors location (Sec. 3.2), candidate subgraphs sampling and augmentation (Sec. 3.3), and pattern collaboration-based graph contrastive learning (Sec. 3.4). Figure 2 presents the framework of the PC-SAD method.

3.1 Problem Formulation

Given a graph $G = (V, E)$, V and E indicate the node set and the edge set, separately. In addition, the node feature matrix $X \in \mathbb{R}^{n \times d}$ contains the node feature information and the adjacency matrix $A \in \mathbb{R}^{n \times n}$ contains the graph structure information, where A is adopted by a novel weighted adjacency matrix [35].

Input G and threshold τ , the unsupervised PC-SAD method first identifies a set of anomaly nodes $N = \{n_1, n_2, \dots, n_m\}$ and samples a set of candidate subgraphs $C = \{c_1, c_2, \dots, c_m\}$ from N . Then, we adopt the augmentation method for candidate subgraphs C to generate positive views $C_p = \{c_{p1}, c_{p2}, \dots, c_{pm}\}$ and negative views

$C_b = \{c_{b1}, c_{b2}, \dots, c_{bm}\}$. These two views are fed into the Pattern Collaboration-based Graph Contrastive Learning to learn a function $f(\cdot)$, which estimates scores S for candidate subgraphs C , where $S = \{s_1, s_2, \dots, s_m\}$. When $s_i > \tau$, the subgraph c_i is considered an anomaly subgraph.

Finally, enter the candidate subgraph C in outlier detector (CO-POD) for classification. The classification results were compared with the detection results to evaluate the performance of PC-SAD.

3.2 Multi-Scale Anchors Location

To locate anomalous and core nodes that are deeply hidden, we propose a Multi-Scale Graph AutoEncoder for anchors detection. This module includes three parts, Encoder, Decoder, and Optimization. Encode generates multi-scale node embeddings by aggregating global and local information. Decoder reconstructs the graph from these embeddings. Optimization component distinguishes anchors by comparing the original and reconstructed graphs.

Encoder. Most Graph AutoEncoder focus on single-scale information, which may limit their capability to capture comprehensive anomalous nodes. Here, Graph Convolutional Network (GCN) and Graph Attention Network (GAT) are adopted to learn the global and local representation of nodes respectively. Then, we design the multi-head feature extraction strategy to highlight hidden anchors.

Inspired by GCN and its success in learning multi-hop neighbor information smoothly through convolution, we utilize a two-layer GCN to aggregate the global structure and feature of the nodes. Specifically, the global node representation H_c can be written as:

$$H_c = GCN(\sigma_1(GCN(A, X; W_c))) \quad (1)$$

where W_c represents the parameter matrix of the 0-th layer, and σ_1 is the activation function, such as ReLU.

In addition to global information, nodes with significant differences in local neighborhoods should also be distinguished. Considering the advantages of GAT in dynamically allocating node weights, we adopt a two-layer GAT to give higher weight for anomaly nodes, formulated as:

$$H_a = GAT(\sigma_1(GAT(A, X; W_a))) \quad (2)$$

where H_a is the representation of local nodes, and W_a is the weight matrix of the 0-th layer.

Then, we aggregate the global and local representation, and apply the MLP to transform the aggregation embedding H_f .

$$H_f = MLP(H_c \oplus H_a) \quad (3)$$

where $\oplus$ is the concatenation operator.

Further, to highlight hidden information, we design the multi head feature extraction strategy, which uses k independent heads to weighted optimization H_f . For the k -th head, the output embedding Y_k is calculated as:

$$\alpha_k = \sigma_2(W_{k2} \cdot \sigma_1(H_f \cdot W_{k1} + b_{k1}) + b_{k2}) \quad (4)$$

$$Y_k = \alpha_k \cdot H_f \quad (5)$$

where, α_k is the feature weight, σ_2 is the activation function, and W_{k1} , W_{k2} , b_{k1} , b_{k2} are trainable parameters.

Afterwards, we concatenate the output embedding of all headers and map them in the same space:

$$Z_f = W_g \cdot [Y_1 \oplus Y_2 \oplus \dots \oplus Y_k] + b_g \quad (6)$$

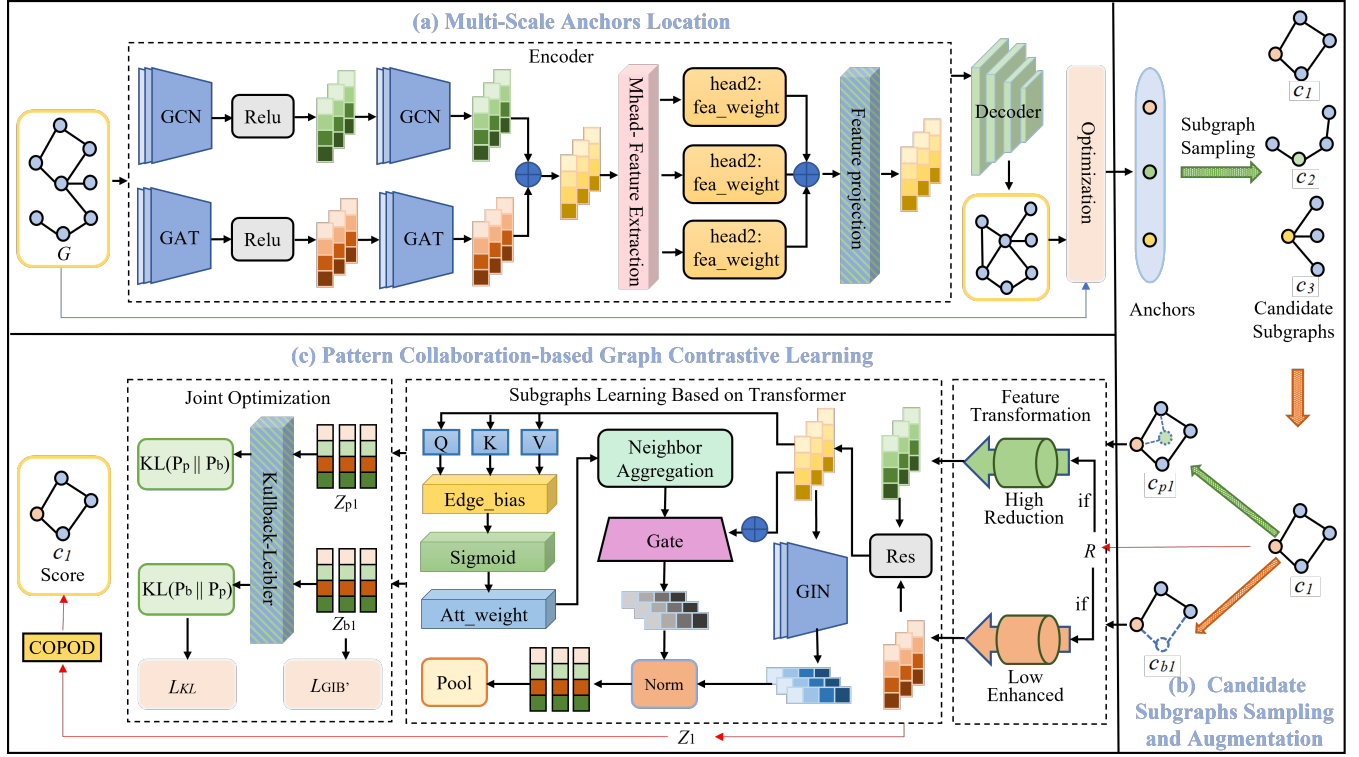


Figure 2: Framework of the PC-SAD.

where W_g, b_g are trainable parameters, Z_f is the final encoding result, which contains multi-scale node information.

Decoder. Upon obtaining the encoding result, we adopt the MLP to reconstruct the node attribute X' and an inner production of fusion embedding to reconstruct the adjacency matrix A' .

$$X' = MLP(Z_f) \quad (7)$$

$$A' = \text{sigmoid}(Z_f \cdot Z_f^T) \quad (8)$$

By decoding, we restore the structure and node attribute of the original graph. This provides error signals for the optimization process.

Optimization. For a node i , a weighted sum of a structure error and an attribute error is used to construct its reconstruction loss L_n , where L_n is written as follows:

$$L_n = \lambda \sum_{j \in Ne(i)} ||X_i - X'_i|| + (1 - \lambda) ||A_{ij} - A'_{ij}|| \quad (9)$$

where $Ne(i)$ is the number of neighboring nodes for node i . $\lambda \in [0, 1]$ is the hyperparameter that is used to adjust the relative importance between structural and attribute errors. Then, minimized L_n to jointly optimize the encoder and decoder.

After optimization, nodes with the larger reconstruction loss are located as anchors and their anomalous score exceeds the threshold value τ . These anomaly nodes form a set $N = \{n_1, n_2, \dots, n_m\}$, where m is the number of anomaly nodes.

3.3 Candidate Subgraphs Sampling and Augmentation

To obtain candidate subgraphs, we start from anchors to sample various structure subgraphs. Then, we generate positive and negative views for each candidate subgraph, which are used to distinguish anomaly subgraphs in contrastive learning.

Candidate Subgraphs Sampling. Usually, the network contains subgraphs of three basic structures. The path structure represents a one-hop connection sequence between nodes; Relationships such as branching or nesting are abstracted into tree structures, representing the hierarchical structures between nodes. The cycle structure represents closed loops of the node, which reflect cyclical patterns of interactions or transactions.

In order to comprehensively capture potential subgraphs, starting from the $N = \{n_1, n_2, \dots, n_m\}$, the Bellman-Ford algorithm [2], the Breadth-First search algorithm [5], and the improved Johnson algorithm [3] are employed to sample these three structures from the initial graph G , respectively. Then, we obtain a set of sampling subgraphs with path structure *Path*, a set of sampling subgraphs with tree structure *Tree*, a set of sampling subgraphs with cycle structure *Cycle*. All subgraphs of these three sets form a set of candidate subgraphs. $C = \{Path, Tree, Cycle\} = \{c_1, c_2, \dots, c_m\}$.

Candidate Subgraphs Augmentation. For $c_i \in Path$, we generate its negative view c_{bi} by dropping the middle node n_i^d , and generate its positive view c_{pi} by adding a new neighbor node n_i^w to the endpoint node n_i^e , the node attribute of n_i^w takes the average

value of the remaining nodes in the path; For $c_i \in Tree$, c_{bi} is generated by dropping the root node n_i^r , and c_{pi} is generated by adding a new child n_i^c to n_i^r , the node attribute of n_i^c takes the average value of the remaining children; For $c_i \in Cycle$, we generate c_{bi} by dropping two random nodes n_i^1 , and n_i^2 , and generate c_{pi} by adding a new node n_i^w both link to n_i^1 , n_i^2 , the node attribute of n_i^w takes the average value of the remaining nodes in the cycle.

Finally, all positive views form a set $C_p = \{c_{p1}, c_{p2}, \dots, c_{pm}\}$ and all negative views form a set $C_b = \{c_{b1}, c_{b2}, \dots, c_{bm}\}$. The positive and negative views are called augmentation subgraphs. The C_p will expand the inherent pattern in the original subgraph by adding nodes and edges, which will share more information with the anomaly subgraphs. The C_b will break this pattern by removing the nodes and edges, which will share less information.

3.4 Pattern Collaboration-based Graph Contrastive Learning

Upon obtaining the augmentation subgraphs, we propose the Pattern Collaboration-based Graph Contrastive Learning module to identify anomalous subgraphs. The module consists of three basic steps: First, we perform feature transformations on subgraphs of different dimensions. Then, the Transformer is introduced to learn the collaborative patterns. Finally, we take the Kullback-Leibler alignment (KL) [6] and Graph Information Bottleneck (GIB) to compare and optimize subgraphs.

Feature Transformation. Considering that different dimensional features will affect the learning effect of subgraphs, we design a dual-space feature transformation component. By calculating the sparsity of the feature matrix, the component adaptively selects the processing path to enhance low-dimensional features and reduce high-dimensional features.

For each augmentation subgraph c_{pi} or c_{bi} , its feature sparsity R is defined as the ratio of non-zero elements in the feature matrix X , which is calculated as:

$$R = 1 - \frac{\text{count}_{\text{non-0}}(X)}{N_g * d_{in}} \quad (10)$$

where N_g represents the number of nodes in an augmentation subgraph, d_{in} is the feature dimension. The smaller R , the larger the dimensions.

Then, two mutually exclusive paths are used to process high or low dimensional features. The path consists of a two-layer linear transformation and a GELU function. When the sparsity of c_{pi} or c_{bi} is less than the threshold δ , it is considered to have high dimensional features. The high-reduction path d_h should be selected in GELU. Conversely, GELU adopts the low-enhanced path d_l .

$$H_t = W_{t2} \cdot \text{GELU}(W_{t1} \cdot X + b_{t1}) + b_{t2} \quad (11)$$

$$\begin{cases} d_h = \min(D, d_{in}/2), & R < \delta \\ d_l = \max(D, 4d_{in}), & R \geq \delta \end{cases} \quad (12)$$

where H_t is the result of the feature transformation. D is an adjustable dimension threshold. W_{t2} , W_{t1} , b_{t1} , b_{t2} are learnable parameters, $W_{t1} \in \mathbb{R}^{d_{in} \times d_l}$ or $\in \mathbb{R}^{d_{in} \times d_h}$, $W_{t2} \in \mathbb{R}^{d_l \times d_{out}}$ or $\in \mathbb{R}^{d_h \times d_{out}}$.

Subgraphs Learning Based on Transformer. To comprehensively explore anomalous collaborative pattern, we adopt Transformer to learn collaborative representation of subgraphs.

First, we project the input using three linear projection matrices to obtain query Q , key K , and value V . Specifically, the $Q(l)$, $K(l)$, and $V(l)$ in l -layer are written as:

$$Q^{(l)} = W_Q^{(l)} \cdot H_t^{(l)} \quad (13)$$

$$K^{(l)} = W_K^{(l)} \cdot H_t^{(l)} \quad (14)$$

$$V^{(l)} = W_V^{(l)} \cdot H_t^{(l)} \quad (15)$$

where $W_Q^{(l)}$, $W_K^{(l)}$, $W_V^{(l)}$ are the learnable parameter of the l -th transformer layer.

To incorporate the collaborative information of the edge, we encode the node-degree feature and add it to the attention weight as a bias. For $e(i, j) \in E$, the attention weight is calculated as:

$$\text{Attention}_{ij} = \text{Softmax}\left(\frac{Q_i^{(l)} K_j^{(l)T}}{\sqrt{d_K}} + \text{Bias}_i\right) \quad (16)$$

$$\text{Bias}_i = W_e \cdot \text{deg}_i + b_e \quad (17)$$

where deg_i is the degree feature of node i , $\sqrt{d_K}$ serves as the scaling factor to prevent the inner product from growing too large.

Based on Attention_{ij} , we aggregate the value vectors of all neighbors of node i :

$$AG_i = \sum_{j \in N(i)} \text{Attention}_{ij} \cdot V_j^{(l)} \quad (18)$$

where AG_i is the aggregated neighbor information. Then, we apply the gate mechanism to obtain results z_s to ensure smooth representation learning.

$$z_{si} = \sigma_1(W_s \cdot [H_i^t \oplus AG_i] + b_s) \quad (19)$$

$$z_s = z_{si} \cdot H_i^t + (1 - z_{si}) \cdot AG_i \quad (20)$$

where W_s , b_s are learnable parameters. In addition to the transformer, we also add a Graph Isomorphism Network (GIN) in the subgraph learning process to increase diversity information.

$$z_n = \text{GIN}(A, H^t; W_n) \quad (21)$$

where z_n is the subgraph representation obtained by GIN. W_n is the learnable parameter. Then, the average pooling function is applied to calculate the average representation of z_s and z_n .

$$Z = \text{pooling}(z_s, z_n) = \frac{z_s + z_n}{2} \quad (22)$$

where Z is final collaborative representation of the augmentation subgraph.

Joint Optimization. Based on final representation of the augmentation subgraph, we design two objective functions to jointly train a model f_θ composed of the parameter set θ , which can efficiently learn embeddings of subgraphs, where $\theta = (W_c, b_{k1}, \dots, b_s)$.

Inspired by the principle of Graph Information Bottleneck, we construct a variant of the contrast loss function $L_{GIB'}$. There, we assume the positive view should share more mutual information with the negative view and less mutual information with other views outside the negative view:

$$\begin{aligned} L_{GIB'} = \min & -\frac{1}{m} \sum_{i=1}^m \psi(f_\theta(Z_{pi}), f_\theta(Z_{bi})) + \\ & \log \frac{1}{m} \sum_{i=1}^m \sum_{j=1, j \neq i}^m e^{\psi(f_\theta(Z_{pi}), f_\theta(Z_{bj}))} \end{aligned} \quad (23)$$

where Z_{pi} is the representation of c_{pi} , Z_{bi} and Z_{bj} are the representation of c_{bi} and c_{bj} . ψ is an optimizer, used by MLP Eq.(23) to identify differences in collaborative patterns.

In addition, due to the inconsistency of the number of nodes in different views, the views will be misaligned, resulting in poor contrast learning. Based on this, we introduce the objective function of KL for probability distributions. The probability distribution of the subgraph $P(c)$ is written as:

$$P(c) = \frac{\exp(c)}{\sum_{k=1}^{d_{pro}} \exp(c_k)} + \epsilon \quad (24)$$

where $\epsilon = 10^{-2}$ is the minimum value to avoid numerical instability caused by a probability of 0. Assume that corresponding probability distribution c_{pi} and c_{bi} are $P_p = P(Z_{pi})$, $P_b = P(Z_{bi})$. To align the distribution difference, the KL is defined as:

$$KL(Z_{pi}, Z_{bi}) = \frac{1}{2} [KL(P_p || P_b) + KL(P_b || P_p)] \quad (25)$$

where $KL(||)$ is calculated as:

$$KL(A || B) = \sum_{k=1}^{d_{pro}} A_k \cdot \log\left(\frac{A_k}{B_k}\right) \quad (26)$$

To achieve mutual constraints between views, the total divergence loss L_{KL} is as follow:

$$L_{KL} = KL(Z_i, Z_{pi}) + KL(Z_i, Z_{bi}) + KL(Z_{pi}, Z_{bi}) \quad (27)$$

where Z_i is the collaborative representation of c_i . Then, L_{KL} is fed with $L_{GIB'}$ to jointly optimize the learn process of the subgraph. The joint loss can be computed as follows:

$$L = \lambda \cdot L_{GIB'} + (1 - \lambda) \cdot L_{KL} \quad (28)$$

After minimizing Eq.(28), the module outputs the collaborative representation of the subgraphs, which can be input to outlier detection (COPOD) or other unsupervised classifiers to be classified.

4 Experiments

4.1 Experiment Setup

Datasets. In experiments, we use four datasets: Ethereum-TSGN (Eth), SimML, Cora-g and CiteSeer-g. Eth is derived from account transaction records on the Ethereum platform. SimML refers to a synthetic dataset for money laundering tasks, which is generated by IBM AMLSim. Cora-g and CiteSeer-g are synthetic datasets from the widely used citation network. Table 1 presents the data details.

Table 1: Details of the four datasets.

Dataset	Node	Edge	Attr	Ano-group	Avg.size
Eth	1823	3254	13	17	7.23
SimML	2768	4226	3123	74	3.52
Cora-g	2708	10358	1433	22	5.68
CiteSeer-g	3327	8924	3703	22	5.14

In Table 1, "Attr", "Ano-group", "Avg.size" are dimensions of node attributes, the number of anomaly subgraphs injected, and average anomaly subgraph size.

Evaluation Metrics. To evaluate PC-SAD's performance, we evaluate anomaly detection from two dimensions: accuracy and completeness. For accuracy, we adopt popular metrics AUC and F1 to evaluate how many subgraphs are correctly classified. For completeness, we introduce the CR metric [1], which takes into account both the number of missing and redundant nodes within a subgraph. CR value approaching 1 indicates a higher level of completeness in the subgraph.

Baselines. We compare the PC-SAD on six state-of-the-art N-SAD or G-SAD methods. The N-SAD includes DOMINANT [16], DeepAE [41], and ComGA [20]. In the N-SAD, we follow the style of AS-GAE [40] by sampling subgraphs from anomaly nodes to complete the task of PC-SAD. The G-SAD includes DeepFD [34], AS-GAE and TP-GrGAD [1]. They first locate the anomalous nodes according to the N-SAD style. Then, the anomaly subgraph is clustering and extracted from around the anomaly node for detection.

4.2 Performance Comparison

Table 2 shows the performance comparison results of PC-SAD and 6 baseline algorithms on 4 datasets. As shown in Table 2, PC-SAD significantly outperforms the N-SAD and G-SAD algorithms in all metrics. Notably with respect to the CR metric, the comparison result of PC-SAD on Cora-g and CiteSeer-g is close to 1, while both N-SAD and some G-SAD algorithms consistently exhibit considerably low CR values. This indicates that baseline algorithms fail to accurately detect anomalous groups, as their prediction groups contain incomplete nodes. These groups cannot be regarded as correctly classified, which leads to low F1 and AUC values.

Although TP-GrGAD is superior to other N-SAD and G-SAD algorithms, there is still a huge gap with PC-SAD. We attribute this to the fact that TP-GrGAD lacks the ability to extract collaborative patterns. It can only aggregate the anomaly scores of neighbor nodes as the subgraphs score. In contrast, PC-SAD can effectively learn graph and subgraph information through Multi-Scale Anchors Location module and Pattern Collaboration-based Graph Contrastive Learning module. This further proves the superiority and effectiveness of PC-SAD in SAD tasks.

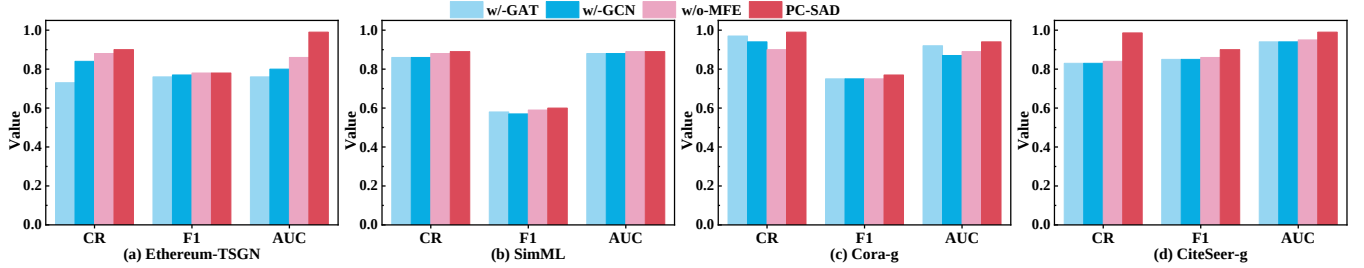
4.3 Ablation Studies of Multi-Scale Anchors Location Module

To evaluate the effectiveness of each component in the Multi-Scale Anchors Location module, we compared PC-SAD with w/-GAT, w/-GCN and w/o-MFE. Here, w/-GAT represents only the local feature extraction component, w/-GCN represents only the global feature extraction component, w/o-MFE eliminates the multi-head feature extraction component from PC-SAD.

Figure 3 shows the performance comparison on different ablation schemes. We can see that PC-SAD consistently outperforms all its components, which indicates that capturing multi-scale information is crucial for effectively distinguishing anchors. When different components are removed, the performance of PC-SAD decreases significantly, $w/-GAT \approx w/-GCN < w/o-MFE$. This proves that each component exhibits significant effectiveness, and only the integration of different components can bring into play the maximum advantage.

Table 2: Comparison results of PC-SAD on all datasets.

Datasets	Metric	DOMINANT	DeepAE	ComGA	DeepFD	AS-GAE	TP-GrGAD	PC-SAD
Eth	CR	0.19±0.01	0.19±0.0	0.18±0.01	0.27±0.04	0.39±0.02	0.81±0.17	0.90±0.01
	F1	0.14±0.01	0.14±0.0	0.14±0.01	0.50±0.09	0.44±0.03	0.73±0.09	0.78±0.0
	AUC	0.51±0.01	0.48±0.01	0.49±0.01	0.52±0.03	0.49±0.09	0.86±0.05	0.99±0.01
SimML	CR	0.33±0.01	0.27±0.03	0.31±0.04	0.12±0.02	0.29±0.01	0.84±0.01	0.89±0.04
	F1	0.45±0.02	0.54±0.03	0.54±0.06	0.37±0.05	0.53±0.04	0.47±0.01	0.60±0.01
	AUC	0.50±0.01	0.49±0.05	0.51±0.05	0.64±0.06	0.56±0.06	0.83±0.02	0.89±0.04
Cora-g	CR	0.42±0.03	0.43±0.04	0.51±0.03	0.38±0.01	0.29±0.02	0.93±0.08	0.99±0.04
	F1	0.45±0.03	0.42±0.02	0.48±0.04	0.42±0.03	0.60±0.06	0.75±0.03	0.77±0.01
	AUC	0.54±0.01	0.50±0.02	0.60±0.05	0.47±0.04	0.57±0.03	0.73±0.01	0.94±0.03
CiteSeer-g	CR	0.39±0.01	0.32±0.04	0.37±0.01	0.30±0.03	0.16±0.02	0.72±0.03	0.99±0.04
	F1	0.42±0.03	0.45±0.04	0.45±0.02	0.58±0.01	0.55±0.01	0.85±0.01	0.90±0.01
	AUC	0.61±0.05	0.55±0.03	0.58±0.08	0.58±0.02	0.59±0.02	0.82±0.02	0.99±0.01

**Figure 3: The performance comparison of the Multi-Scale Anchors Detection module.**

4.4 Ablation Studies of Pattern Collaboration Based Graph Contrastive Learning Module

In order to explore the effect of each component in the Pattern Collaboration-based Graph Contrastive Learning module, we carried out the following ablation experiments. w/-DSFT: it retains the “Feature Transformation” component; w/o-Trans: it eliminates the Transformer component, where the module is directly concatenated through the DSFT component and the GIN component; w/o-GIN: it eliminates the GIN component, where the module is concatenated through the DSFT component and the Transformer component.

Figure 4 shows the performance comparison on different ablation schemes. As shown in Figure 4, although there is no significant difference between the three components in detecting anomaly subgraphs, when combined, PC-SAD outperforms other components on most datasets. This observation confirms that three components are suitable for this module and can effectively learn collaborative pattern. However, on the SimML, the performance of PC-SAD is close to other components. We attribute this to the adversarial samples in SimML. Specifically, SimML causes normal transactions to be misclassified as anomalies through minor perturbations, thereby reducing the detection performance of PC-SAD.

4.5 Visualization

To demonstrate that PC-SAD can effectively learn collaborative patterns for distinguishing subgraphs, we use t-SNE [23] to visualize in Figure 5 the embeddings of subgraphs by the module of Pattern Collaboration-based Graph Contrastive Learning. Despite a few unsuccessful cases, such as in Figure 5 (b), most embeddings of the same class are prone to clustering. This means that PC-SAD learn high-quality embeddings and can accurately capture anomalous collaboration structures. For example, the embeddings of the anomaly subgraphs (red nodes) in Figure 5 (d) are close to each other, while they are far removed from those of the normal subgraphs (green nodes). We can also observe a similar situation in Figure 5 (c) and Figure 5 (a).

4.6 Parameter Sensitivity of τ

In previous research, a widely adopted approach involves considering the top- $p\%$ of nodes with the highest scores as anomalous nodes. This approach defines τ as the p -th percentile. We follow this approach and show experimental results of PC-SAD under different p settings in Figure 6. Different p values had a significant effect on PC-SAD, which was about 0.05-0.22. Especially on SimML, the

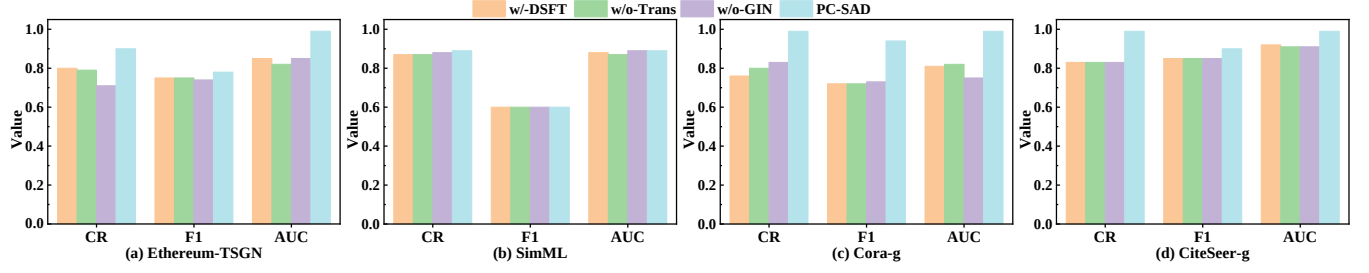


Figure 4: The performance comparison of the Pattern Collaboration-based Graph Contrastive Learning module.

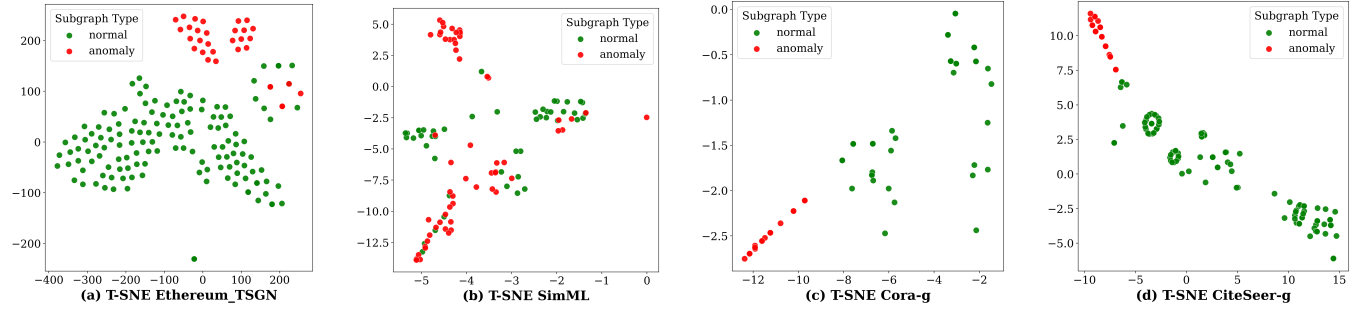


Figure 5: Visualizations on the embeddings of subgraphs.

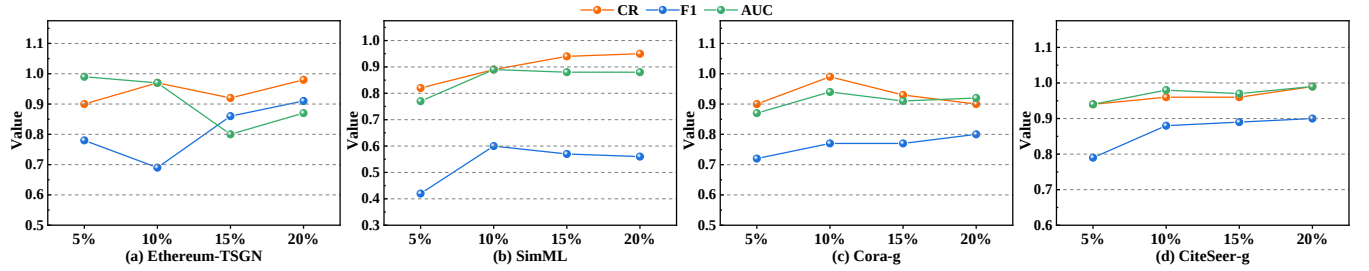


Figure 6: Parameter sensitivity of τ .

difference between the three metrics is 0.11-0.18. This confirmed that the value of τ/p are sensitive to PC-SAD and require strict control of parameter accuracy. Finally, We chose the optimal p as the overall experimental results of PC-SAD, where 5% for Eth, 10% for SimML and Cora-g, 20% for CiteSeer-g.

5 Conclusion

In this paper, we first investigate the limitations of Subgraph Anomaly Detection methods based on group level from the perspective of pattern collaboration, revealing that prior work performs poorly in locating a subgraph and learning collaborative representation. To address the above challenges, we propose a new unsupervised method PC-SAD. It identifies the anomaly nodes through Multi-Scale Graph AutoEncode and locates candidate subgraphs through sampling and enhancement methods. Finally, the method of Pattern Collaboration-based Graph Contrastive Learning is employed to learn collaborative embeddings of candidate subgraphs, thereby

distinguishing anomaly subgraphs. Experimental results on four benchmark datasets demonstrate that PC-SAD outperforms competitors and effectively detects anomaly gangs in financial fraud.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grants U24B20153, 62572205, 62272182, 62302183 and by the Hubei Provincial Natural Science Foundation of China under Grant 2025AFD762. This work was also supported by the Open Research Fund of The State Key Laboratory of Blockchain and Data Security, Zhejiang University (Grant A2554), Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education of China (Grant KLCS20240401), and CCF-DiDi GAIA Collaborative Research Funds (Grant CCF-DiDi GAIA 202522).

References

- [1] Xing Ai, Jialong Zhou, Yulin Zhu, Gaolei Li, Tomasz P Michalak, Xiapu Luo, and Kai Zhou. 2024. Graph anomaly detection at group level: A topology pattern enhanced unsupervised approach. In *2024 IEEE 40th International Conference on Data Engineering*. 1213–1227.
- [2] Richard Bellman. 1958. On a routing problem. *Quarterly of applied mathematics* 16, 1 (1958), 87–90.
- [3] Etienne Birmelé, Rui Ferreira, Roberto Grossi, Andrea Marino, Nadia Pisanti, Romeo Rizzi, and Gustavo Sacomoto. 2013. Optimal listing of cycles and st-paths in undirected graphs. In *Proceedings of the twenty-fourth annual ACM-SIAM symposium on Discrete algorithms*. 1884–1896.
- [4] Belgün Ergenç Bostanoğlu and Nourhan Abuzayed. 2024. Dynamic frequent subgraph mining algorithms over evolving graphs: a survey. *PeerJ Computer Science* 10 (2024), e2361.
- [5] Alan Bundy and Lincoln Wallen. 1984. Breadth-first search. In *Catalogue of artificial intelligence tools*. 13–13.
- [6] Dong Chen, Xiang Zhao, and Weidong Xiao. 2024. Fine-grained anomaly detection on dynamic graphs via attention alignment. In *2024 IEEE 40th International Conference on Data Engineering*. 3178–3190.
- [7] Tianyi Chen and Charalampos Tsourakakis. 2022. Antibenford subgraphs: Unsupervised anomaly detection in financial networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2762–2770.
- [8] Christos Cholevas, Eftychia Angeli, Zacharoula Sereti, Emmanouil Mavrikos, and George E Tsekouras. 2024. Anomaly detection in blockchain networks using unsupervised learning: A survey. *Algorithms* 17, 5 (2024), 201.
- [9] Kaize Ding, Jundong Li, Rohit Bhanushali, and Huan Liu. 2019. Deep anomaly detection on attributed networks. In *Proceedings of the 2019 SIAM international conference on data mining*. 594–602.
- [10] Jingcan Duan, Siwei Wang, Pei Zhang, En Zhu, Jingtao Hu, Hu Jin, Yue Liu, and Zhibin Dong. 2023. Graph anomaly detection via multi-scale contrastive learning networks with augmented view. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 7459–7467.
- [11] Yuan Gao, Xiang Wang, Xiangnan He, Zhengguang Liu, Huamin Feng, and Yongdong Zhang. 2023. Addressing heterophily in graph anomaly detection: A perspective of graph spectrum. In *Proceedings of the ACM web conference 2023*. 1528–1538.
- [12] Jingtao Hu, Siwei Wang, Jingcan Duan, Hu Jin, Xinwang Liu, and En Zhu. 2025. Higher-order Enhanced Contrastive-based Graph Anomaly Detection Without Graph Augmentation. *Pattern Recognition* (2025), 111666.
- [13] Jingtao Hu, Bin Xiao, Hu Jin, Jingcan Duan, Siwei Wang, Zhao Lv, Siqi Wang, Xinwang Liu, and En Zhu. 2025. SAMCL: Subgraph-Aligned Multiview Contrastive Learning for Graph Anomaly Detection. *IEEE Transactions on Neural Networks and Learning Systems* 36, 1 (2025), 1664–1676.
- [14] Zhang Junyu. 2024. Simple Graph Anomaly Detection via Multi-Scale Contrastive Learning Networks. In *2024 IEEE 6th International Conference on Power, Intelligent Computing and Systems*. 75–81.
- [15] Xiangjie Kong, Jin Liu, Huan Li, Chenwei Zhang, Jiaxin Du, Dongyan Guo, and Guojiang Shen. 2025. Graph Anomaly Detection via Diffusion Enhanced Multi-View Contrastive Learning. *Knowledge-Based Systems* 311 (2025), 113093.
- [16] Meng-Chieh Lee, Yue Zhao, Aluna Wang, Pierre Jinghong Liang, Leman Akoglu, Vincent S Tseng, and Christos Faloutsos. 2020. Autoaudit: Mining accounting and time-evolving graphs. In *2020 IEEE International Conference on Big Data*. 950–956.
- [17] Fu Lin, Yue Zhang, Xuexiong Luo, Mingkan Li, Zitong Wang, and Enfeng Song. 2025. Multi-view graph anomaly detection via subgraph anomaly augmentation. *Neurocomputing* 637 (2025), 130109.
- [18] Jiayi Liu, Changchun Yin, Hao Wang, Xiaofei Wu, Dongwan Lan, Lu Zhou, and Chungpeng Ge. 2023. Graph embedding-based money laundering detection for Ethereum. *Electronics* 12, 14 (2023), 3180.
- [19] Zhengyang Liu, Hang Yu, and Xiangfeng Luo. 2025. Federated Graph Anomaly Detection via Disentangled Representation Learning. In *Proceedings of the ACM on Web Conference 2025*. 1216–1224.
- [20] Xuexiong Luo, Jia Wu, Amin Beheshti, Jian Yang, Xiankun Zhang, Yuan Wang, and Shan Xue. 2022. Comga: Community-aware attributed graph anomaly detection. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 657–665.
- [21] Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z Sheng, Hui Xiong, and Leman Akoglu. 2021. A comprehensive survey on graph anomaly detection with deep learning. *IEEE transactions on knowledge and data engineering* 35, 12 (2021), 12012–12038.
- [22] Xiaoxiao Ma, Jia Wu, Jian Yang, and Quan Z Sheng. 2023. Towards graph-level anomaly detection via deep evolutionary mapping. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 1631–1642.
- [23] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9 (2008), 2579–2605.
- [24] Shuxin Qin, Yongcan Luo, Jing Zhu, Gaofeng Tao, Jingya Zheng, and Zhongjun Ma. 2024. Anomaly Detection on Attributed Networks via Multiview and Multi-scale Contrastive Learning. *IEEE Transactions on Computational Social Systems* (2024).
- [25] Jing Ren, Feng Xia, Ivan Lee, Azadeh Noori Hoshyar, and Charu Aggarwal. 2023. Graph learning for anomaly analytics: Algorithms, applications, and challenges. *ACM Transactions on Intelligent Systems and Technology* 14, 2 (2023), 1–29.
- [26] Durgesh Samariya and Amit Thakkar. 2023. A comprehensive survey of anomaly detection algorithms. *Annals of Data Science* 10, 3 (2023), 829–850.
- [27] Prakash Shelokar, Arnaud Quirin, and Oscar Cordon. 2013. MOSubdue: a Pareto dominance-based multiobjective Subdue algorithm for frequent subgraph mining. *Knowledge and information systems* 34, 1 (2013), 75–108.
- [28] Chuancheng Song, Xixun Lin, Hanyang Shen, Yanmin Shang, and Yanan Cao. 2025. Uniform: Towards unified framework for anomaly detection on graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12559–12567.
- [29] Ying Sun. 2024. Implicit anomaly subgraph detection (IASD) in multi-domain attribute networks. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 8510–8511.
- [30] Ying Sun, Wenjun Wang, Nannan Wu, and Chunlong Bao. 2024. Anomaly subgraph detection through high-order sampling contrastive learning. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 2362–2369.
- [31] Susheel Suresh, Pan Li, Cong Hao, and Jennifer Neville. 2021. Adversarial graph augmentation to improve graph contrastive learning. *Advances in Neural Information Processing Systems* 34 (2021), 15920–15933.
- [32] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep graph infomax. In *Proceedings of the International Conference on Learning Representations* (2019).
- [33] Tomer Voronov, Danny Raz, and Ori Rottenstreich. 2023. A framework for anomaly detection in blockchain networks with sketches. *IEEE/ACM Transactions on Networking* 32, 1 (2023), 686–698.
- [34] Haibo Wang, Chuan Zhou, Jia Wu, Weizhen Dang, Xingquan Zhu, and Jilong Wang. 2018. Deep structure learning for fraud detection. In *2018 IEEE international conference on data mining*. 567–576.
- [35] Xiaofeng Wang, Shuaiming Lai, Shuailei Zhu, Yuntao Chen, Laishui Lv, and Yuanqian Qi. 2024. Graph Anomaly Detection via Multiscale Contrastive Self-Supervised Learning From Local to Global. *IEEE Transactions on Computational Social Systems* (2024).
- [36] Asiri Wijesinghe and Qing Wang. 2022. A new perspective on "how graph neural networks go beyond weisfeiler-lehman?". In *International conference on learning representations*.
- [37] Jiayang Wu, Wensheng Gan, Jiahao Zhang, and Philip S Yu. 2025. Contrastive learning for anomaly detection in hierarchical subgraph. *Knowledge-Based Systems* 317 (2025), 113435.
- [38] Yiming Xu, Zhen Peng, Bin Shi, Xu Hua, Bo Dong, Song Wang, and Chen Chen. 2025. Revisiting Graph Contrastive Learning on Anomaly Detection: A Structural Imbalance Perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12972–12980.
- [39] Junchi Yu, Jie Cao, and Ran He. 2022. Improving subgraph recognition with variational graph information bottleneck. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 19396–19405.
- [40] Zheng Zhang and Liang Zhao. 2022. Unsupervised deep subgraph anomaly detection. In *2022 IEEE International Conference on Data Mining*. 753–762.
- [41] Dali Zhu, Yuchen Ma, and Yinlong Liu. 2020. Anomaly detection with deep graph autoencoders on attributed networks. In *2020 IEEE Symposium on Computers and Communications*. 1–6.
- [42] Zhong Zhuang, Kai Ming Ting, Guansong Pang, and Shuaibin Song. 2023. Subgraph centralization: A necessary step for graph anomaly detection. In *Proceedings of the 2023 SIAM International Conference on Data Mining*. 703–711.